

Airbnb Clone

SDE Fullstack Assignment — Step-by-Step Execution Guide

Next.js (TypeScript) + FastAPI + SQLite — Written like a senior engineer would plan it: schema first, API second, UI third, deploy early, polish last.

Format — Read this once end-to-end before writing code. Then work section by section — each one is self-contained and gives you exact commands, schema, and code you can adapt.

Contents

1. Strategy & Golden Rules
2. Tech Stack Decisions
3. Project Structure
4. Database Schema Design
5. Backend — Step by Step (FastAPI)
6. Seed Data Strategy
7. Frontend — Step by Step (Next.js)
8. Suggested Time Budget
9. Deployment Guide
10. README Template
11. Preparing for the Evaluation Interview
12. Common Pitfalls to Avoid
13. Bonus Features (If Time Remains)

1. Strategy & Golden Rules

This is a breadth assignment, not a depth assignment. The rubric rewards a complete, working, persistent, deployed app over a partially-finished pixel-perfect one. Optimize for finishing the full loop (search → book → host manages it) before polishing anything.

Five rules that will save you the most time

- 1 Schema first. Every hour spent on the DB schema before writing a single endpoint saves three hours of migration pain later. Lock it in Section 4 before touching code.
- 2 Backend before frontend, but deploy both on day one. Stand up a hello-world FastAPI on Render and a hello-world Next.js on Vercel talking to each other before building real features. Deployment bugs found on hour 20 are the #1 cause of missed deadlines.
- 3 Seed data is a feature, not a chore. The assignment explicitly grades this. Write the seed script in parallel with your models, not at the end — it doubles as your manual test data.
- 4 Mock aggressively, exactly where the brief tells you to. Auth, payments, messaging, identity verification — all explicitly allowed to be fake. Do not spend real time here.
- 5 Commit and redeploy often (every 60–90 minutes). Small, frequent deploys catch environment drift immediately instead of compounding into a hour-long debugging session at the end.

Time reality check — The brief estimates ~24 hours. Treat that as the must-have scope, not a hard budget — most candidates land closer to 26–30 hours once environment setup and deployment friction are included. Section 8 breaks this down phase by phase.

2. Tech Stack Decisions

The brief fixes the big three (Next.js/TS, FastAPI or Django, SQLite). Everything below fills the gaps with choices that are fast to build with and easy to defend in the interview.

Layer	Choice	Why
Frontend framework	Next.js 14, App Router, TypeScript	File-based routing gives you <code>/listing/[id]</code> and <code>/host/listings</code> for free; Server Components reduce client JS for the listing grid.
Styling / UI	Tailwind CSS + shadcn/ui	shadcn gives you accessible dialog, calendar, toast and card primitives you can restyle to look like Airbnb instead of hand-rolling modals.
Date-range picker	react-day-picker	Native range-select mode plus a <code>`disabled`</code> prop that takes an array of date ranges — maps directly onto your booked-dates API response.
Client data fetching	TanStack Query (React Query)	Caching + refetch-on-focus means booking a date range instantly reflects in the calendar without manual state plumbing.
Client/global state	Zustand	Tiny store for wishlist + mock session — avoids Redux boilerplate for a 24-hour build.
Backend framework	FastAPI	Async, automatic OpenAPI/Swagger docs (huge for the interview — you can demo every endpoint live), Pydantic validation for free.
ORM	SQLAlchemy 2.0 (declarative) + Pydantic v2 schemas	Clean separation between DB models and API response shapes; easy to explain relationships.
Migrations	Alembic (optional) or <code>`Base.metadata.create_all()`</code> on startup	For a 24h build, <code>`create_all()`</code> at startup plus a seed script is faster and defensible — mention Alembic as the production-grade alternative in your README.

Layer	Choice	Why
Auth	Mocked — simple email/password row check + a signed session cookie or dummy JWT	Brief explicitly allows simplified auth. Don't build real password hashing flows, refresh tokens, etc. One `/auth/login` that returns a token is enough.
Images	Hot-linked stock photo URLs (Unsplash/Picsum), stored as plain URL strings	Brief explicitly allows this. Real upload-to-cloud-storage is a bonus item only.
Map	Static image or Leaflet + OpenStreetMap (free, no API key)	Leaflet is a ~30 min bonus add if core features are done; otherwise a static image with a pin overlay satisfies the requirement.

3. Project Structure

One repo, two top-level folders — matches the deliverable requirement exactly ("a public GitHub repository containing frontend/ and backend/").

backend/

```

backend/
  app/
    main.py                # FastAPI app, CORS, router includes
    config.py              # settings (DB url, CORS origins) via pydantic-settings
    database.py            # SQLAlchemy engine + session dependency
    models/
      user.py  listing.py  booking.py  review.py  amenity.py  wishlist.py
    schemas/              # Pydantic request/response models, mirrors models/
    routers/
      auth.py  listings.py  bookings.py  wishlist.py  reviews.py  host.py
    services/
      availability.py      # overlap-checking logic, lives outside the router
    seed.py               # idempotent seed script, also run on startup if DB empty
  requirements.txt
  Dockerfile             # optional, useful for Render/Railway
  .env.example

```

frontend/

```

frontend/
  app/
    layout.tsx  page.tsx                # Home / Explore
    listing/[id]/page.tsx                # Listing detail
    trips/page.tsx                        # My Trips (guest)
    wishlist/page.tsx
  host/
    page.tsx                             # Host dashboard (owned listings)
    listings/new/page.tsx                # Create listing
    listings/[id]/edit/page.tsx         # Edit listing
  components/
    ListingCard.tsx  SearchBar.tsx  FilterBar.tsx  Gallery.tsx
    DateRangePicker.tsx  BookingSummary.tsx  ReviewList.tsx
    Navbar.tsx  ListingForm.tsx  WishlistButton.tsx
  lib/
    api.ts                # thin fetch wrapper, reads NEXT_PUBLIC_API_URL
    types.ts              # shared TS types matching backend schemas
    store.ts              # Zustand store (session, wishlist)
  .env.local.example

```

4. Database Schema Design

This is one of the explicitly graded criteria ("Database Design — well-structured schema with proper relationships"), so spend real thought here before writing models. Eight tables cover every must-have feature with no redundant modelling.

Entity relationships

- users (1) — (N) listings — a user becomes a host simply by owning listings; no separate host table.
- users (1) — (N) bookings — a user is the guest on a booking.
- listings (1) — (N) listing_photos, (1) — (N) bookings, (1) — (N) reviews
- listings (N) — (N) amenities via the join table listing_amenities
- users (N) — (N) listings via wishlists (a guest can wishlist many listings)

Table definitions

Table	Key columns	Notes
users	id PK, name, email UNIQUE, password_hash, is_host BOOL, avatar_url, created_at	Any user can flip is_host=true and start listing — matches the brief's simplified guest-vs-host notion without a separate host entity.
listings	id PK, host_id FK→users, title, description, property_type, city, country, lat, lng, price_per_night DEC, max_guests, bedrooms, beds, bathrooms, avg_rating FLOAT, is_active BOOL, created_at, updated_at	avg_rating is denormalized (recomputed on new review) so the listing grid can sort/display ratings without a JOIN on every request.
listing_photos	id PK, listing_id FK, url, sort_order INT	Ordered gallery. sort_order 0 = cover photo shown on the listing card.
amenities	id PK, name UNIQUE	Seed once: Wifi, Kitchen, Free parking, Pool, AC, Washer, Workspace, TV, Pet friendly, Self check-in.
listing_amenities	listing_id FK, amenity_id FK, PK(listing_id, amenity_id)	Pure join table — enables the amenities filter with a simple IN/JOIN query.
bookings	id PK, listing_id FK, guest_id FK→users, check_in DATE, check_out DATE, guests_count INT, nightly_rate_snapshot DEC, total_price DEC, status (confirmed/cancelled/completed), created_at	nightly_rate_snapshot freezes the price at booking time — the listing price can change later without rewriting history. status drives My Trips vs cancelled filtering.
reviews	id PK, listing_id FK, guest_id FK, booking_id FK NULLABLE, rating INT(1-5), comment TEXT, created_at	booking_id links a review to a completed stay (bonus: "leave a review after a completed stay") but stays nullable so seed reviews don't need a matching booking.
wishlists	id PK, user_id FK, listing_id FK, UNIQUE(user_id, listing_id), created_at	Composite uniqueness stops duplicate favorites at the DB layer, not just in the UI.

SQLAlchemy model — the two trickiest tables

```
# models/booking.py
class Booking(Base):
    __tablename__ = "bookings"
    id = Column(Integer, primary_key=True)
    listing_id = Column(Integer, ForeignKey("listings.id"), nullable=False)
    guest_id = Column(Integer, ForeignKey("users.id"), nullable=False)
    check_in = Column(Date, nullable=False)
    check_out = Column(Date, nullable=False)
    guests_count = Column(Integer, nullable=False)
    nightly_rate_snapshot = Column(Numeric(10, 2), nullable=False)
    total_price = Column(Numeric(10, 2), nullable=False)
    status = Column(Enum("confirmed", "cancelled", "completed", name="booking_status"),
                    default="confirmed")
    created_at = Column(DateTime, server_default=func.now())

    listing = relationship("Listing", back_populates="bookings")
    guest = relationship("User", back_populates="bookings")

    __table_args__ = (
        CheckConstraint("check_out > check_in", name="ck_checkout_after_checkin"),
    )
```

The one query that matters most: overlap validation

This is the core booking-integrity rule the brief calls out ("no overlapping/unavailable dates"). Two date ranges overlap if, and only if, one starts before the other ends — on both sides:

```
# services/availability.py
def has_overlap(db: Session, listing_id: int, check_in: date, check_out: date,
               exclude_booking_id: int | None = None) -> bool:
    q = db.query(Booking).filter(
        Booking.listing_id == listing_id,
        Booking.status == "confirmed",
        Booking.check_in < check_out,      # existing starts before new one ends
        Booking.check_out > check_in,      # existing ends after new one starts
    )
    if exclude_booking_id:
        q = q.filter(Booking.id != exclude_booking_id)
    return db.query(q.exists()).scalar()

# in the create-booking endpoint:
if has_overlap(db, payload.listing_id, payload.check_in, payload.check_out):
    raise HTTPException(409, "Selected dates are no longer available")
```

Be ready to explain this — This exact half-open-interval overlap check is the kind of thing interviewers ask you to whiteboard. Know why `check_in < check_out` on one side and `check_out > check_in` on the other is required (it correctly handles back-to-back bookings where one checkout date equals another's check-in date).

5. Backend — Step by Step (FastAPI)

Step 5.1 — Scaffold

```
mkdir backend && cd backend
python -m venv .venv && source .venv/bin/activate
pip install fastapi "uvicorn[standard]" sqlalchemy pydantic pydantic-settings \
    python-multipart passlib[bcrypt] python-jose alembic
pip freeze > requirements.txt
```

Wire up `main.py` with CORS open to your Vercel domain + localhost, and include routers as you write them. Run with: `uvicorn app.main:app --reload` — Swagger UI is free at `/docs`.

Step 5.2 — Models & schemas

Write all eight SQLAlchemy models from Section 4 first, in one sitting. Then write matching Pydantic schemas (ListingRead, ListingCreate, ListingUpdate, BookingCreate, BookingRead, etc.) — keep Create/Update schemas separate from Read schemas so you never accidentally expose password_hash in an API response.

Step 5.3 — Database bootstrap

```
# database.py
engine = create_engine("sqlite:///./app.db", connect_args={"check_same_thread": False})
SessionLocal = sessionmaker(bind=engine, autoflush=False)

# main.py (startup)
@app.on_event("startup")
def on_startup():
    Base.metadata.create_all(bind=engine)
    from app.seed import run_seed_if_empty
    run_seed_if_empty() # safe to call on every boot – see Section 6
```

Step 5.4 — API surface

Build endpoints in this order: listings (read-only) → seed data → bookings → host CRUD → wishlist/reviews/auth last, since those are the smallest.

Method & path	Purpose
GET /listings	Search + filter + paginate. Query params: location, check_in, check_out, guests, min_price, max_price, property_type, amenities, page, page_size
GET /listings/{id}	Full detail: photos, amenities, host info, avg_rating
GET /listings/{id}/availability	Booked date ranges, for disabling dates in the calendar
POST /listings	Create (host only)
PUT /listings/{id}	Update (owner check: listing.host_id == current_user.id)
DELETE /listings/{id}	Soft delete (is_active = false) so past bookings stay intact
POST /bookings	Validate overlap + guest count ≤ max_guests, then create
GET /bookings/me	My Trips — current user's bookings, newest first
GET /listings/{id}/bookings	Host dashboard — bookings for one owned listing
PATCH /bookings/{id}/cancel	Cancel a booking (frees the dates)
POST /listings/{id}/reviews	Add review, then recompute avg_rating
GET /listings/{id}/reviews	List reviews for a listing
POST /wishlist/{listing_id}	Add to wishlist
DELETE /wishlist/{listing_id}	Remove from wishlist
GET /wishlist/me	Current user's wishlist
GET /amenities	For populating the filter checkboxes
POST /auth/register • POST /auth/login • GET /auth/me	Mocked auth — see callout below

Mocking auth without cutting corners — Store a real bcrypt-hashed password and issue a real signed JWT on login — it's five extra lines with python-jose and makes the 'mocked/simplified auth' claim in your README credible in the interview, instead of a hardcoded fake user.

6. Seed Data Strategy

Explicitly called out in the brief: "Seed a variety of listings (with photos), a few users acting as hosts, and some existing bookings so the app is immediately usable." Treat seed.py as a deliverable, not a throwaway script.

What to generate

- 3–4 host users, each owning 4–6 listings, across 5–6 different cities/countries so location search is meaningfully testable.
- 15–20 listings total spanning multiple property_types (apartment, house, cabin, villa) and a wide price spread (■1,500–■25,000/night equivalent) so price-range filtering has something to bite.
- 3–5 photos per listing, in a curated list of ~40 real Unsplash CDN image URLs (interior/exterior shots) reused across listings by index — far more convincing than random picsum.photos noise.
- 8–10 amenities, attached in varying combinations so the amenities filter isn't trivial.
- 8–12 bookings: mix of past (status=completed), current, and future (status=confirmed) dates, across different listings and guests — this is what makes the calendar and My Trips page look real on first load instead of empty.
- 5–10 reviews tied to the completed bookings, with varied ratings, so avg_rating differs across listings.

Make it idempotent

Guard the seed so re-deploys (which reset an ephemeral SQLite file — see Section 9) don't duplicate or crash on unique-constraint errors:

```
def run_seed_if_empty():
    db = SessionLocal()
    try:
        if db.query(User).count() > 0:
            return # already seeded
        hosts = [User(name=..., email=..., is_host=True) for ... in HOSTS]
        db.add_all(hosts); db.flush()
        listings = [Listing(host_id=h.id, ...) for h, data in zip(hosts, LISTINGS)]
        db.add_all(listings); db.flush()
        # photos, amenities, bookings, reviews follow the same add_all + flush pattern
        db.commit()
    finally:
        db.close()
```

7. Frontend — Step by Step (Next.js)

Step 7.1 — Scaffold

```
npm create-next-app@latest frontend --typescript --tailwind --eslint --app
cd frontend
npm shadcn@latest init
npm shadcn@latest add button card dialog calendar toast avatar badge \
  dropdown-menu input select checkbox skeleton
npm install @tanstack/react-query zustand react-day-picker date-fns lucide-react
```

Step 7.2 — Layout & navigation

- Sticky Navbar: logo, pill-shaped search bar (collapses to a compact version on scroll — the signature Airbnb interaction, worth doing since it's the single highest-recognition UI cue), 'Become a host' link, avatar/profile dropdown with mocked guest↔host switch.
- Wrap the app in a `QueryClientProvider` (React Query) at the root layout.

Step 7.3 — Home / Explore page

- Category/filter row: horizontally scrollable pills (Cabins, Trending, Amazing views, ...) — purely cosmetic filters on `property_type` are fine, they don't all need real backing logic.
- Listing grid: responsive CSS grid (1 col mobile → 4-5 cols desktop), each `ListingCard` shows cover photo (with a heart/wishlist toggle overlaid top-right), location, dates if searched, price/night, and a star rating pulled from `avg_rating`.
- Search bar drives location + `check_in/check_out` + guests as URL query params — this makes the page shareable/bookmarkable and lets the grid re-fetch via React Query keyed on those params.
- Filter row (price range slider, property type, amenities checkboxes) opens in a shadow Dialog or Sheet, same pattern as Airbnb's 'Filters' button.
- Pagination or infinite scroll: infinite scroll via `IntersectionObserver` + React Query's `useInfiniteQuery` is more visually impressive and isn't much extra code over classic pagination.

Step 7.4 — Listing detail page

- Photo gallery: a 1-large + 4-small CSS grid that opens a full Dialog carousel on click — this is the single most recognizable Airbnb layout pattern, prioritize it over anything else visual.
- Title, host card (avatar, name, 'Host since ...'), description, amenities list with icons (lucide-react has icons for wifi, parking, pool, etc.).
- Date-range picker: `react-day-picker` in range mode; fetch `GET /listings/{id}/availability` and pass the booked ranges into the `'disabled'` prop so unavailable dates are visually blocked.
- Price breakdown card: nights × nightly rate, + cleaning fee (flat mocked value), + service fee (e.g. 12% of subtotal) = total — recompute live as the date range changes.
- Reviews section: average rating + count at the top, individual review cards below.

Step 7.5 — Booking flow

- 1 Guest selects dates + guest count on the listing page → 'Reserve' button enables once a valid range is picked and `guests_count ≤ max_guests`.
- 2 Booking summary panel (can be the same price-breakdown card, now sticky) shows the final total.
- 3 'Confirm and pay' opens a mocked checkout — a Dialog with fake card fields (no real validation needed beyond basic format) and a 1–2 second fake loading state for realism.
- 4 On confirm, `POST /bookings`. On 409 (overlap), surface a toast: 'Those dates were just booked — please pick another range' and refetch availability.
- 5 On success, show a confirmation screen/toast and redirect to `/trips`.
- 6 `/trips` (My Trips) lists the guest's bookings from `GET /bookings/me`, grouped into Upcoming / Past, each linking back to the listing.

Step 7.6 — Host dashboard (CRUD)

- `/host` — table or card grid of the current host's listings (`GET /host/listings`), each row showing status, booking count, and Edit/Delete actions.
- `/host/listings/new` and `/host/listings/{id}/edit` share one `ListingForm` component: title, description, property type select, price, max guests, bedrooms/beds/bathrooms, location, amenities checkboxes, and a repeatable photo-URL input list.
- Delete should soft-delete via the API and optimistically remove the row from the table (React Query mutation + `invalidateQueries`).
- Each listing row expands (or links) to its bookings via `GET /listings/{id}/bookings`.

Step 7.7 — Wishlist, toasts, polish

- Heart icon on every ListingCard toggles POST/DELETE /wishlist/{id}; keep it optimistic in Zustand so it feels instant, reconciled against the server response.
- /wishlist page reuses the same ListingCard grid, filtered to saved listings.
- Toasts (shadcn/sonner) on: booking confirmed, booking failed, listing saved/created/deleted, review submitted — cheap to add, disproportionately improves the 'feels like a real product' score.
- Loading states: skeleton cards (shadcn Skeleton) instead of spinners for the grid and gallery.
- Responsive pass last: Tailwind's grid/flex utilities mean most of this is free if you built mobile-first from the start — don't leave it as a separate 'make it responsive' task.

8. Suggested Time Budget

Roughly 28 hours all-in, spread across 3 sessions — realistic buffer over the brief's 24h estimate. Deploy at the end of Session 1 even though features are incomplete: it de-risks the final submission entirely.

Session	Phase	Hours
Session 1 — Foundation	Repo + tech setup, DB schema locked	2.5 h
	Backend scaffold: models, schemas, DB bootstrap	2 h
	Listings API (search/filter/paginate) + seed script v1	3 h
	Deploy skeleton backend (Render) + skeleton frontend (Vercel), confirm they talk	1.5 h
Session 2 — Core features	Bookings API + overlap validation + host CRUD API	2.5 h
	Wishlist, reviews, mocked auth API	1.5 h
	Frontend: layout, navbar, Home/Explore grid + search + filters	4 h
	Frontend: Listing detail page (gallery, calendar, price breakdown)	3 h
Session 3 — Booking loop, host UI, ship	Booking flow + My Trips	2.5 h
	Host dashboard CRUD UI	2.5 h
	Wishlist UI, toasts, responsive pass, dark mode if time allows	2 h
	Final seed data pass (variety + volume)	1 h
	Deployment hardening + smoke test full flow end-to-end	1.5 h
	README + architecture writeup	1.5 h
	Buffer for the inevitable surprise bug	1 h

9. Deployment Guide

Step 9.1 — Backend on Render (or Railway)

- 1 New Web Service → connect the GitHub repo → set root directory to backend/.
- 2 Build command: `pip install -r requirements.txt`
- 3 Start command: `uvicorn app.main:app --host 0.0.0.0 --port $PORT`
- 4 Env vars: `CORS_ORIGINS=https://<your-vercel-domain>.vercel.app,http://localhost:3000`
- 5 Deploy, then confirm /docs loads and /listings returns seeded data.

SQLite persistence gotcha — Render/Railway free-tier web services use an ephemeral filesystem — the `app.db` file can reset on redeploy or dyno restart. Because your startup hook auto-reseeds an empty DB (Section 6), the app is always usable, and data created during a live evaluation session persists for the life of that running instance. If you want data to survive redeploys too, attach a persistent disk (Render paid plans) or a volume (Railway) mounted at the SQLite file's path — mention this tradeoff explicitly in your README; it reads as an informed engineering decision, not an oversight.

Step 9.2 — Frontend on Vercel

- 1 Import the repo → set root directory to `frontend/`.
- 2 Env var: `NEXT_PUBLIC_API_URL=https://<your-render-backend>.onrender.com`
- 3 Default build command/output detection works out of the box for Next.js.
- 4 Deploy, then click through the full flow once on the live URL: search → listing → book → My Trips → host dashboard → create listing → delete listing.

Step 9.3 — CORS

```
# main.py
app.add_middleware(
    CORSMiddleware,
    allow_origins=settings.cors_origins,    # from env var, split on comma
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

10. README Template

All four required sections (setup, tech stack, architecture, DB schema, API overview) in one copy-paste starting point — fill in the bracketed parts.

```
# Airbnb Clone

## Tech Stack
Frontend: Next.js 14 (App Router, TypeScript), Tailwind CSS, shadcn/ui, React Query, Zustand
Backend: FastAPI, SQLAlchemy 2.0, Pydantic v2, SQLite
Deploy: Frontend on Vercel, Backend on Render

## Live Demo
Frontend: https://[your-app].vercel.app
API docs: https://[your-api].onrender.com/docs

## Architecture Overview
[1 short paragraph + optional diagram: Next.js client -> REST API (FastAPI) -> SQLite,
 mention where auth/session lives, where images are served from (hot-linked URLs)]

## Database Schema
[Paste the table from Section 4 of your planning doc, or link schema.sql / models/]

## API Overview
[Paste the endpoint table from Section 5]

## Local Setup

### Backend
cd backend
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
cp .env.example .env
```

```
uvicorn app.main:app --reload
# -> API on http://localhost:8000, docs on /docs, auto-seeds on first run

### Frontend
cd frontend
npm install
cp .env.local.example .env.local # set NEXT_PUBLIC_API_URL=http://localhost:8000
npm run dev
# -> http://localhost:3000

## Assumptions & Simplifications
- Authentication is simplified: [describe exactly what's real vs mocked]
- Payments are fully mocked, no real payment gateway is integrated
- Images are hot-linked stock URLs rather than uploaded to cloud storage
- SQLite is ephemeral on the free hosting tier; the app auto-seeds on startup if empty
- [add any other decisions you made under time pressure]

## Bonus Features Implemented
[list whichever of: interactive map / post-stay reviews / superhost badges /
image upload / dark mode / full responsive – you actually built]
```

11. Preparing for the Evaluation Interview

The rubric explicitly grades "Code Understanding — ability to explain your code during evaluation." AI-assisted code is fine per the brief, but you need to own every decision. Rehearse these out loud, not just mentally:

Be ready to walk through one feature end-to-end

Pick the booking flow. Practice narrating it in under 3 minutes: the click on 'Reserve' in the UI → the POST /bookings request → the Pydantic validation → the overlap query in services/availability.py → the DB write → the response → the toast → the My Trips refetch. Interviewers use this to gauge whether you understand your own system or just assembled AI output.

Questions you should pre-answer for yourself

- Why SQLite and not Postgres for this? (Brief's requirement; mention you'd swap the connection string for Postgres in production — SQLAlchemy makes that a one-line change.)
- Why is avg_rating denormalized on the listing table instead of computed with a JOIN each time?
- Walk through the overlap-check query and explain the half-open interval logic.
- Why did you soft-delete listings instead of hard-deleting them?
- What's actually mocked vs real in your auth flow, and what would you change to make it production-ready?
- How does the frontend know which dates are already booked, end to end?
- If you had another day, what would you build next?

What NOT to do

- Don't memorize code you didn't understand while pasting it in — if a reviewer asks 'why this line' and you can't answer, that's the exact failure mode the brief warns about.
- Don't over-claim bonus features in the README that aren't actually wired up end-to-end — reviewers will click them.

12. Common Pitfalls to Avoid

Pitfall	Fix
Chasing pixel-perfect Airbnb UI before core flows work	Get search → detail → book → host-CRUD fully wired with plain shadcn styling first. Visual polish is one grading criterion out of seven — functionality gates all the others.
Building real image upload / cloud storage early	Explicitly a bonus. URL-string photos satisfy every must-have. Only revisit if everything else is done with hours to spare.
Deploying for the first time near the deadline	Deploy a hello-world version of both services in the first session (Section 8). Most 'it works on my machine' failures are CORS/env-var issues that are cheap to fix early and expensive to fix under deadline pressure.
Weak or sparse seed data	It's graded directly and it's also your test harness — an empty or repetitive-looking app makes every other feature harder to demo, including to yourself while building.
Real auth complexity (refresh tokens, email verification, OAuth)	Explicitly out of scope. A single hashed-password login returning a JWT is enough.
No overlap validation, or client-side-only validation	Must be enforced server-side (Section 4) — a client bug or a second browser tab must not be able to create a double-booking.
Skipping the README until the last 20 minutes	It's a listed deliverable with its own grading weight. Fill in the template from Section 10 incrementally as you build, not at the end.

13. Bonus Features (If Time Remains)

Only start these once every must-have item works end-to-end and is deployed. Ordered by effort-to-impact ratio, cheapest first:

Bonus	Effort	Notes
Full responsive design	Low	Near-free if you built mobile-first with Tailwind from the start — mostly a QA pass at 375px/768px/1280px widths.
Dark mode	Low	Tailwind's dark: variant + a theme toggle in the navbar using shadcn's pattern; maybe 30–45 minutes.
Superhost badges / rating aggregation	Low	avg_rating is already denormalized (Section 4) — a badge is just a conditional render where avg_rating ≥ 4.8 and review_count ≥ 5.
Leave a review after a completed stay	Medium	Gate the 'Write a review' button on the guest having a booking with status=completed and check_out in the past for that listing.
Interactive map with listing pins	Medium	Leaflet + OpenStreetMap tiles, no API key needed; plot listings using the lat/lng columns already in the schema.
Image upload to cloud storage	High	Lowest priority — explicitly optional and the least connected to the core booking/CRUD flows being evaluated.

Good luck. Ship the full loop first, polish second — that ordering is what this whole guide is optimized for.